

WERKGROEP API HISTORIE · KENNISPLATFORM APIs

Historie in gegevens

Het grensvlak van de API — ontdekkingen, patronen en open vraagstukken

Rob van Dort · data architect

2 september 2026 · Geonovum, Amersfoort

Voortbouwend op de kick-off van 1 juli 2026

Waarom deze werkgroep

- De overgang van **StUF / Digikoppeling-ebMS** naar API-opvolgers is in volle gang
- De huidige overheids-API-standaarden vertonen **gaten** ten opzichte van StUF-ebMS — juist rond **mutaties, historie, tijdreizen en verantwoording**
- Concrete trekkers: **LV WOZ** (adviestraject), **LV BGT** (nieuwe mutatie-interface nodig op korte termijn), **Federatief Datastelsel, Uit Betrouwbare Bron**
- Rol van de werkgroep: het **grensvlak** invullen — hóe werkt gegevensuitwisseling via API's, en welke **generieke bouwstenen** zijn eruit te lichten?

Tijdreizen / historie:

een generiek probleem, steeds specifiek opgelost

- Voor het eerst met deze complexe problematiek geconfronteerd in **2004** (Delta Lloyd verzekeringen)
- Daarna bij **UWV, KVK, Kadaster, IND, CIBG, UBB, FDS/DbdB, ...**
- Telkens **maatwerk** — meestal Java + relationele database
- Vaak **hetzelfde patroon**, maar opnieuw uitgevonden (soms full snapshot: NS/ProRail)
- Het ontwikkelen van engine-achtige functionaliteit in project-context is gedoemd te falen
- Kostbaar (onverwachte complexiteit), kwetsbaar (half uitgedachte / uitgewerkte oplossing)
- **AI** verhoogt de urgentie: betere feiten → betrouwbaardere antwoorden — en feiten **veranderen** in de tijd
- Conclusie: een **generiek probleem** verdient een **generieke oplossing**
- Werk in uitvoering: **DigiTwinDB**
 - Bitemporele Object Graph engine op basis van een immutable append-only event log

Focus & insteek

- Nadruk op het **grensvlak** (de API), niet op de internals
- Maar toch ook een blik **onder de motorkap** — geeft beter beeld op grensvlak
- Verhaallijn: **ontdekkingen** · **terugkerende patronen** · **open vraagstukken**
- Kernaspecten: **mutaties, queries, historie en verantwoording**
- Context: 'losse' **API, Landelijke Voorziening, Data Product, Federatief Datastelsel**
- Praktische insteek: een case. Wat zou kunnen werken? Voer voor latere discussie & uitwerking

Graag kritisch interactief, maar let op de tijd (45 minuten in deze agenda)



01

Context 1

Case: mini-register “GeoPol”

Actuele data over gemeentelijke indelingen beschikbaar gesteld als API.

Uitgangspunten

- Gestructureerde data, objectmodel
- Data worden centraal bijgehouden via mutatie-API, beschikbaar gesteld via query-API
- Demo-domein: **GeoPol** — Country, Province, Municipality, City
- Vergelijk PDOK Bestuurlijke Gebieden, maar dan meer data gericht:
<https://www.pdok.nl/introductie/-/article/bestuurlijke-gebieden>
- Compact domein, herkenbaar, niet triviaal (business logica, tijdsaspect)
- Miniatuur dataset: gemeente Wijchen / Heumen van 1800 tot nu
- Volledige dataset: heel NL vanaf 1800 tot nu (Work in Progress)
- Geen uitgebreide theoretische verhandeling vooraf
- Gewoon volgen van *een* implementatie

Karakteristiek van een bitemporele Object Graph engine

- **Logisch model** — het domein **zónder tijdsaspecten**, zoals de gebruiker het kent:
 - **UML Class Diagram** — de 'dingen' met hun attributen en relaties, sterk getypeerd
- **Technisch model** — mét tijdsaspecten: versies, geldigheid, registratie
 - in generiek zin aanwezig in de engine: de domein types mappen hier naar
- **Bitemporeel** — twee tijdassen:
 - transTime** (wanneer geregistreerd)
 - validTime** (wanneer geldig in de werkelijkheid)
- **Immutable event store** — niets wordt overschreven, elke mutatie is set nieuwe feiten;
- **Onderscheid Entity en EntitySnapshot**
 - Entity**: het object met alle attribuut-versies (multi-dimensionaal object)
 - EntitySnapshot**: een **projectie** van het object naar transTime, ValidTime

Twée interfaces, verschillende eisen

Mutatie-interface — schrijven

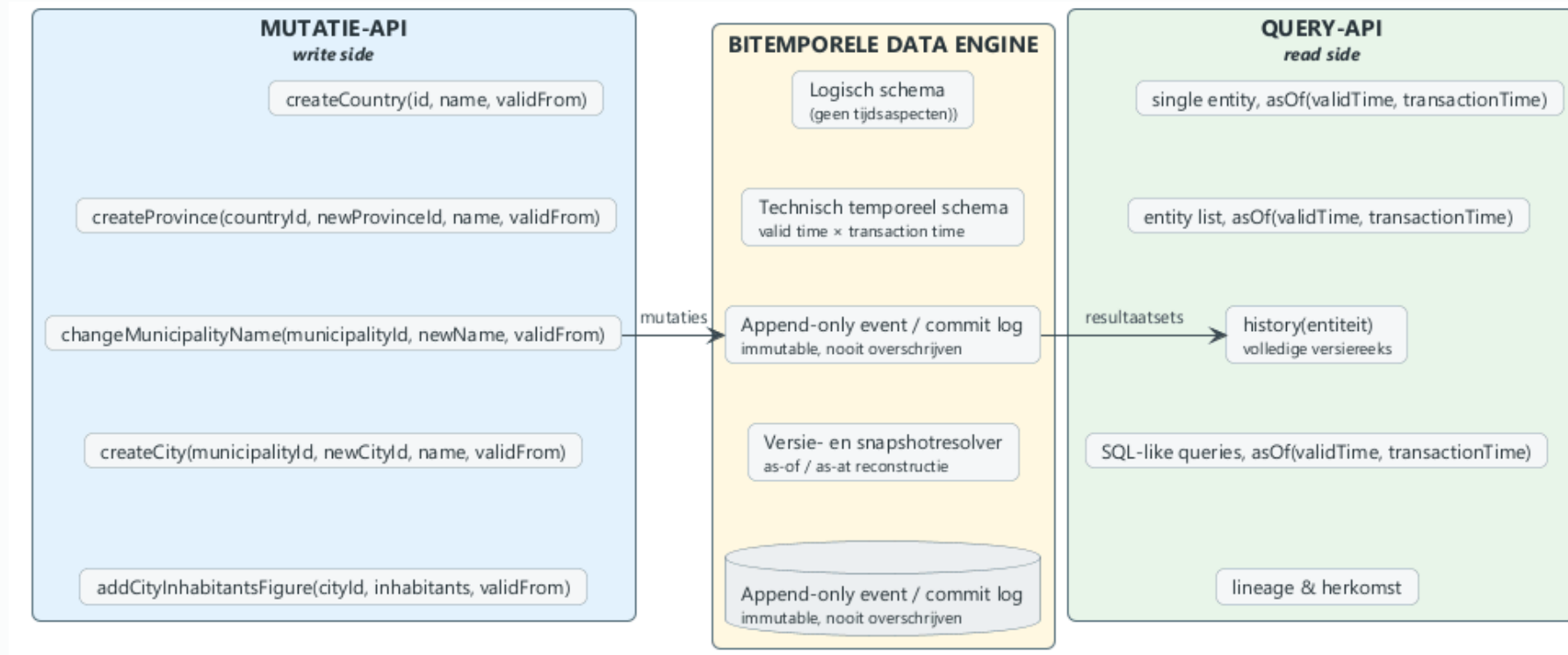
Laag volume, hoge eisen aan consistentie. Domeingedreven (DDD) met een Anti-Corruption Layer.

Read-interface — lezen

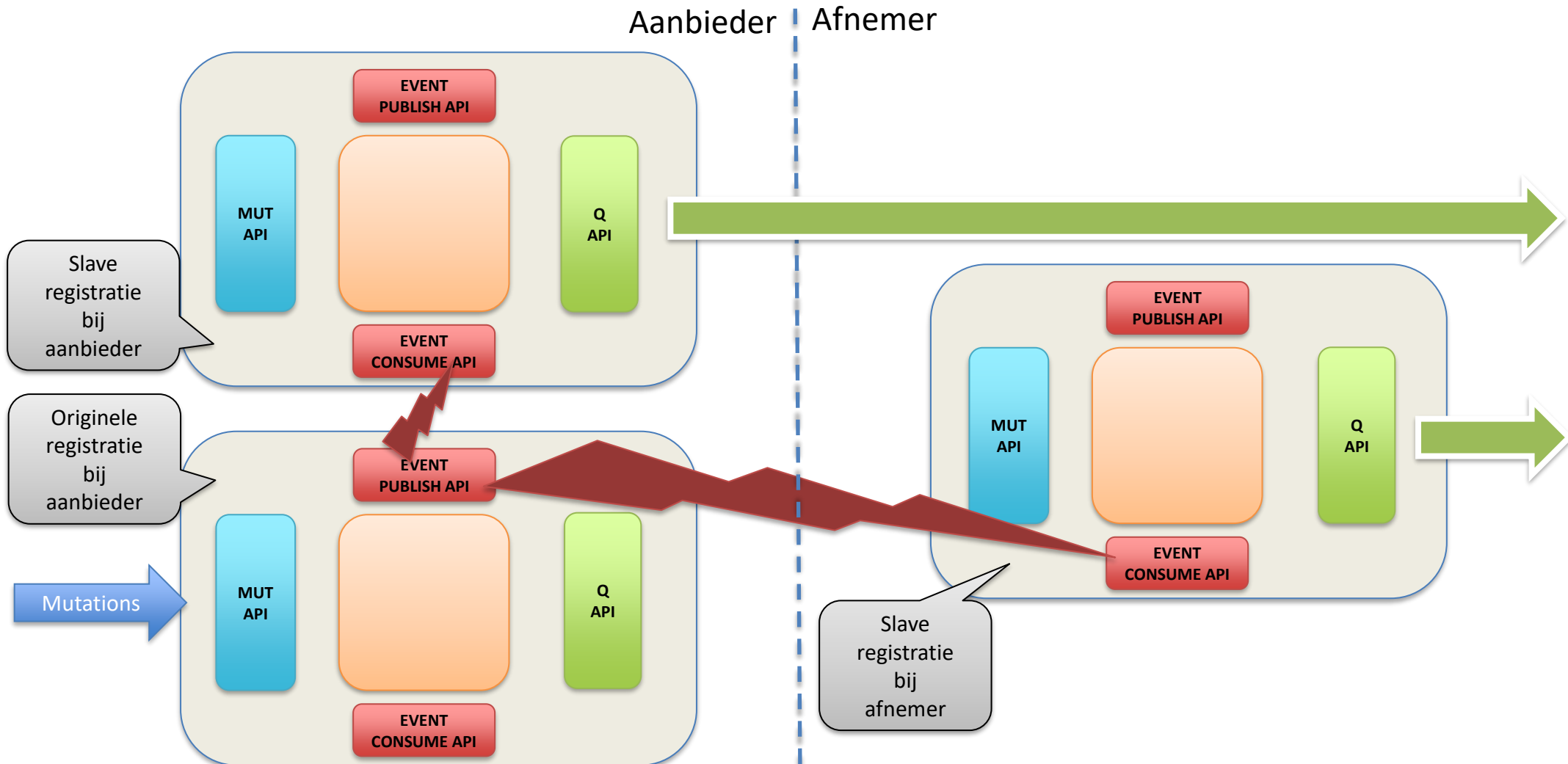
Hoog volume, high-performance, multi-modaal.
Bitemporele parameters over de hele graph.

- Cross-platform standaard: **GraphQL** — schema, gerichte selectie over de graph, streaming
- **REST** in principe minder geschikt: dataset bevragingen zijn beperkt resource-georiënteerd
- Lees-modaliteiten: **GraphQL Streaming** · **Apache Arrow** · **Arrow Flight** · **ADBC**
- **Bijzonderheid: RPC remote object model**

Data-engine: motorblok met twee interfaces



Extreme schaalbaarheid door append-only, immutable event log





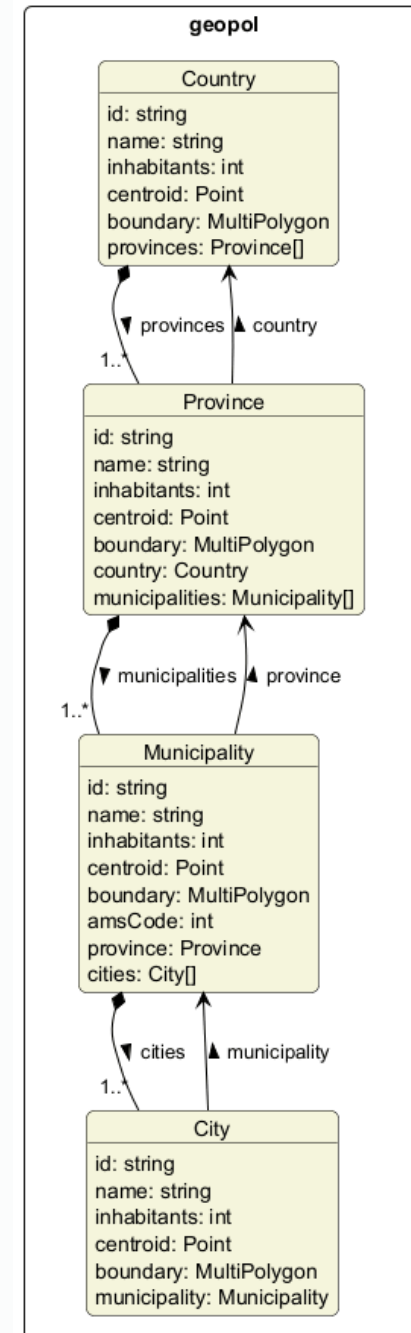
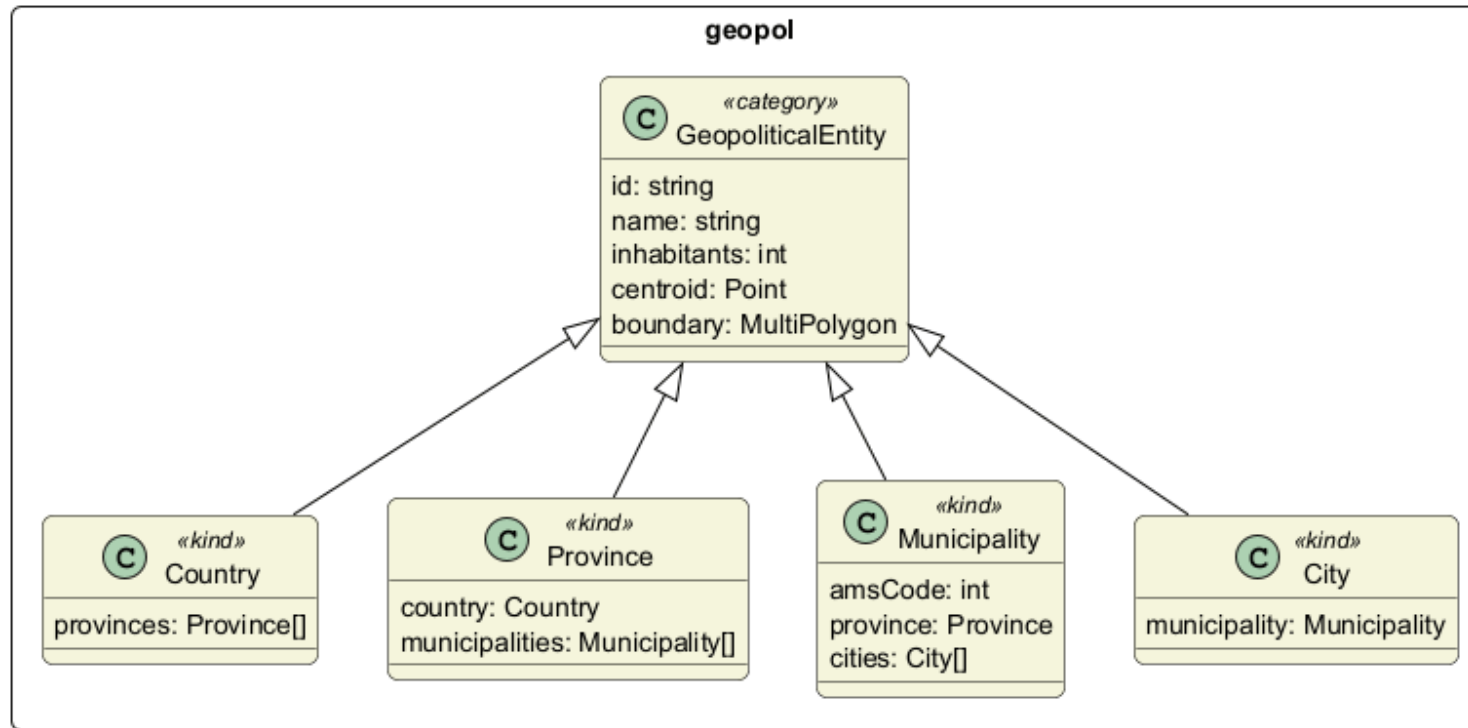
02

HET GRENSVLAK — DE API

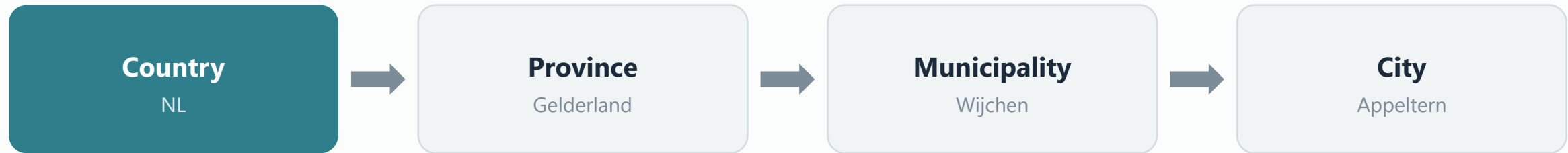
Mutatie- en query-interface

Concreet gemaakt met het GeoPol-demodomein: Land · Provincie · Gemeente · Woonplaats.

Gemeentelijke indeling als demo-domein



Gemeentelijke indeling als demo



elke laag "hangt af van" de laag erboven — volgorde doet ertoe bij create-operaties

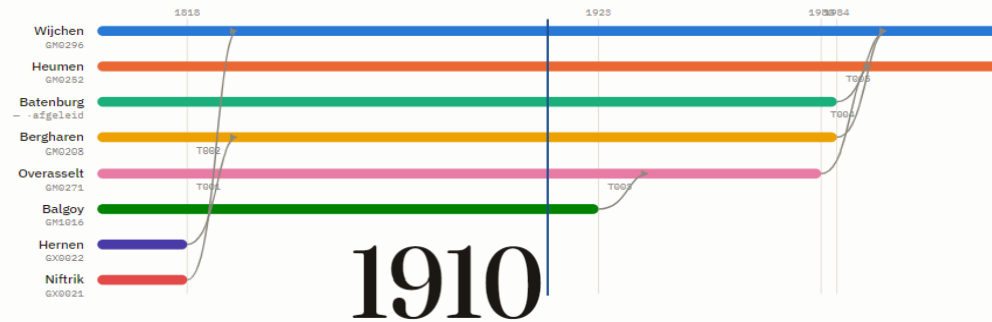
- Nederland **1812–2026**: honderden herindelingen, splitsingen, hernoemingen, provinciewissels (gemiddeld 4–5 grenswijzigingen per jaar)
- **Geen officieel temporeel register** — CBS publiceert jaarlijks, platgeslagen naar 1 januari
- Klein genoeg om te tonen, rijk genoeg om **alle patronen** te raken

HET DEMO DOMEIN

Maas en Waal uitgelicht

Fusieboom

Elke gemeente loopt van 1795 tot haar opheffing; de pijl wijst naar de opvolger. De verticale lijn is het gekozen jaar.

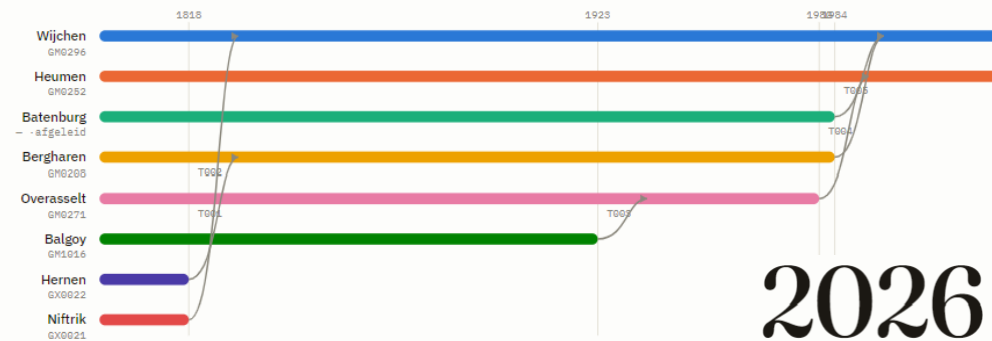


1910

Beweeg over een balk voor de herindelingstransactie.

Fusieboom

Elke gemeente loopt van 1795 tot haar opheffing; de pijl wijst naar de opvolger. De verticale lijn is het gekozen jaar.



2026

Beweeg over een balk voor de herindelingstransactie.

Tijdreizen door het Land van Maas en Waal

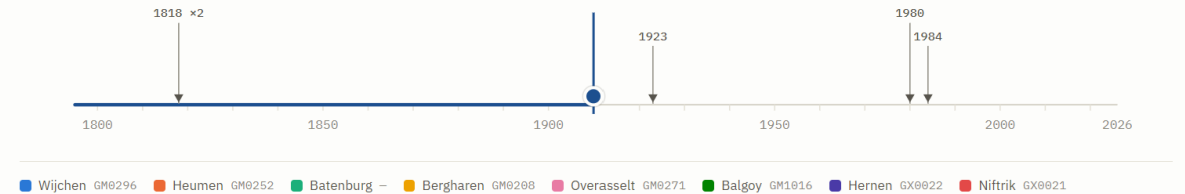
Sleep door de geldigheidstijd van 1795 tot 2026 en zie hoe woonplaatsen van gemeente wisselen terwijl hun inwonertal meebeweegt. Vier herindelingen persen acht zelfstandige gemeenten samen tot drie.

1910

ZELFSTANDIGE GEMEENTEN
6

TIJDPAK
na T001 + T002 · 1818

▶ Speel af 1x 3x ↺ 1795



Bestuurlijke indeling

op 1 januari 1910

LAND

Nederland

PROVINCIE

Gelderland

GEMEENTEN & WOONPLAATSEN

Wijchen GM0296 2 wpl · 33.004 inw.

Wijchen 32.654 Niftrik 350

Bergharen GM0208 3 wpl · 1.633 inw.

Bergharen 841 Hernen 601

Leur 191

Overasselt GM0271 1 wpl · 1.477 inw.

Overasselt 1.477

Heumen GM0252 1 wpl · 1.347 inw.

Heumen 1.347

Batenburg - afgeleid 1 wpl · 540 inw.

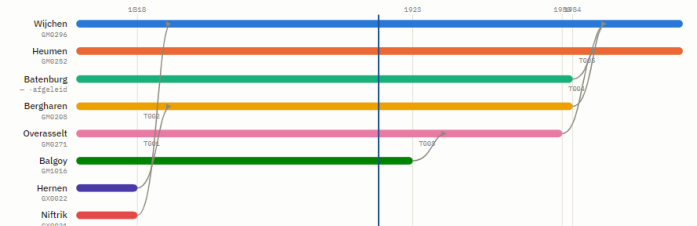
Batenburg 540

Balgoy GM1016 1 wpl · 481 inw.

Balgoy 481

Fusieboom

Elke gemeente loopt van 1795 tot haar opheffing; de pijl wijst naar de opvolger. De verticale lijn is het gekozen jaar.



Beweeg over een balk voor de herindelingstransactie.

Muteren volgens Domain Driven Design

- **Anti-Corruption Layer**: de buitenwereld praat domeintaal; de ACL bewaakt dataconsistentie en handelt events af
- Een **create-operatie per entiteittype** — en respecteer de afhankelijkheden (Country vóór Province vóór Municipality vóór City)
- **validTime** is optioneel op te geven bij elke mutatie
- **transTime** wordt door het systeem bepaald op het moment van registreren
- Mutaties zijn **additief** — nieuw feit (beter: feit-versie), nooit een overschrijving

GeoPol — de mutatie-operaties

```
createCountry(id, name, validTime)
createProvince(countryId, id, name, validTime)
createMunicipality(provinceId, id, name, validTime)
createCity(municipalityId, id, name, validTime)

moveCity(cityId, toMunicipalityId, validTime)
mergeMunicipality(fromMunicipalityId, toMunicipalityId, validTime)

... renameMunicipality · splitMunicipality · correct... · withdraw... · end...
```

Elk werkwoord heeft een eigen, expliciet effect op de tijdlijnen. transTime wordt niet meegegeven.

Registreren ≠ corrigeren ≠ terugtrekken ≠ beëindigen

- **Registreren** — nieuw feit; nieuwe validTime-periode
- **Corrigeren** — het feit was fout vastgelegd; herstel binnen dezelfde validTime, nieuwe transTime
- **Terugtrekken / herroepen** — het feit had er nooit moeten staan; effect verdwijnt, het spoor blijft
- **Beëindigen** — het feit was geldig maar is dat niet meer; de validTime krijgt een einde
- “Corrigeren verschilt fundamenteel van terugtrekken” — deze scheiding stuurt de complexiteit
 - Open vraagstuk: één gestandaardiseerde vocabulaire hiervoor
 - **Transactie** — set logisch bij elkaar horende nieuwe feiten

Bevragen met (optioneel) transTime en validTime

- **transTime opgegeven** → effecten van transacties ná transTime worden weggefilterd — elimineert mutaties/correcties met terugwerkende kracht
 - default: álle tot nog toe geregistreerde effecten tellen mee (“zoals we het nu weten”)
- **validTime opgegeven** → het effect van een mutatie is pas zichtbaar vanaf die validTime
- Één paar (transTime, validTime) geldt voor **álle objecten in dezelfde query** → consistente momentopname over de hele graph
- Beide weglaten = de huidige stand, zoals nu bekend

Drie vragen aan dezelfde data — voorbeeld Wijchen

- “Hoe was de gemeentelijke indeling **op 1 januari 1850?**” → validTime = 1850-01-01
- “Wat **dachten we bij de load** dat de indeling in 1850 was?” → transTime = load-datum, validTime = 1850-01-01
- “Wat is er **sindsdien met terugwerkende kracht** gecorrigeerd voor 1850?” → verschil tussen twee transTime-sneden
- Zonder bitemporeel model zijn vraag 2 en 3 niet te beantwoorden

Elke waarde herleidbaar, elk antwoord reproduceerbaar

- Immutable event store = **volledige keten**: van een getoonde waarde terug naar de transactie die haar veroorzaakte
- Scheid “**waarom**” van “**hoe**” — de motivatie van de medewerker staat los van hoe het daarna is vastgelegd (wijziging ≠ geldigheid, correctie ≠ formele historie)
- **Reproduceerbaarheid**: dezelfde query + hetzelfde (transTime, validTime) → gegarandeerd hetzelfde antwoord, ook over jaren
- Hoe gaat StUF om met historie? API-standaard moet dit **expliciet** borgen



03

TERUGKERENDE PATRONEN

Wat we in elk project weer tegenkomen

Dezelfde thema's, of het nu verzekeringen, UWV, KvK, Kadaster, ??? is.

Patronen die telkens terugkomen

- **Twee tijdlijnen zijn het minimum** — in de praktijk vaak meer (formeel/registratie, materieel, juridisch)
- **Out-of-order** — gegevens komen niet in chronologische volgorde binnen
- **Correctie of nieuwe versie?** — nieuw ID of nieuw versienummer na een wijziging: de API dwingt hier een keuze af
- **Event vs. state** — gebeurtenis-tijd waaruit de toestand volgt, of toestand met een geldigheidsperiode



04

OPEN VRAAGSTUKKEN

Waar we samen uit moeten komen

Binnen het domein van API's en gegevensuitwisseling — zonder in het “ultieme historiemodel” te verdwalen.

De vragen voor de werkgroep

- Hoeveel tijdassen zijn genoeg — **twee**, of in de praktijk **drie tot vijf**?
- Correctie vs. terugtrekken/herroepen — in **één gestandaardiseerde vorm**?
- Hoe **abstract** kan het grensvlak zijn zónder de datastructuur voor te schrijven? (identificatie & versionering dwingen keuzes af)
- **Synchronisatie** als vierde pijler naast onderhoud / bevragen / verantwoording? (bijv. CloudEvents)
- Welke **bindings** standaardiseren — REST, GraphQL, Arrow Flight?
- Aansluiting op **Federatief Datastelsel, Uit Betrouwbare Bron**

Voorstel voor de aanpak

- Begin met een “**synchronisation of minds**”: gedeelde begrippen, definities, een licht metamodel
- **Prioriteit: onderhoud** (mutatie-interface) boven bevragen — LV BGT heeft die op korte termijn nodig
- **Bouw op bestaande componenten** — en kies expliciet wat we loslaten (conflicterende StUF/NEN-delen)
- **Implementeerbaar vandaag** — geen big-bang-transitie vereisen
- **Starterdocument** — ontdekkingen · patronen · kandidaat-antwoorden · open vragen

Om te onthouden

- Historie in gegevens is een **generiek, terugkerend** probleem — het hoort op het grensvlak thuis, niet in elk project opnieuw
- **Bitemporeel** (transTime + validTime) + **immutable events** + **DDD-mutatie-interface** = een beheersbaar model
- De API-standaard moet **tijdreizen, herleidbaarheid en reproduceerbaarheid** expliciet ondersteunen
- De **werkwoorden** — registreren, corrigeren, terugtrekken, beëindigen — zijn de kern van de mutatie-interface



Vragen & discussie

Volgende stap in deze sessie: prioriteren en de eerste stap uitlijnen; ik werk het starterdocument uit.

Meedoen of meelezen: api@geonovum.nl

Concept & demo: DigiTwinDB (GeoPol)